1. Aim

To demonstrate thread synchronization in Java by allowing multiple customers to book tickets while safely updating the shared seat count.

2. Description

The TicketCounter starts with 10 seats. Three customer threads—Ravi (4 tickets), Priya (3 tickets), and Rahul (5 tickets)—try to book seats. The synchronized bookTicket() method lets only one thread at a time check and update the shared seat count. Thread execution order can vary.

3. Software Requirements

Hardware: Computer or laptop with at least 2 GB RAM recommended.
Operating system: Windows, Linux, or macOS.
Software: JDK 8 or later and a text editor or Java IDE.
File name: TicketBooking.java. Keep it in the recordprograms package folder.

4. UML Diagram

Creates one counter and three customer threads.

5. Java Code

Save as **TicketBooking.java**.

```
package recordprograms;

class TicketCounter
{
    private int seats = 10;

    public synchronized void bookTicket(String name, int count)
    {
        System.out.println(name + " requested " + count + " tickets.");

        if (count <= seats)
        {
            seats = seats - count;

            System.out.println("Booking successful for " + name);
            System.out.println("Tickets booked: " + count);
            System.out.println("Seats remaining: " + seats);
        }
        else
        {
            System.out.println("Booking failed for " + name);
            System.out.println("Not enough seats available.");
        }
    }
}

class Customer extends Thread
{
    TicketCounter counter;
    int tickets;

    Customer(TicketCounter counter, String name, int tickets)
    {
        super(name);
        this.counter = counter;
        this.tickets = tickets;
```

```
    }

    public void run()
    {
        counter.bookTicket(getName(), tickets);
    }
}

public class TicketBooking
{
    public static void main(String[] args) throws InterruptedException
    {
        TicketCounter counter = new TicketCounter();

        Customer c1 = new Customer(counter, "Ravi", 4);
        Customer c2 = new Customer(counter, "Priya", 3);
        Customer c3 = new Customer(counter, "Rahul", 5);

        c1.start();
        c2.start();
        c3.start();

        c1.join();
        c2.join();
        c3.join();

        System.out.println("Ticket booking completed.");
    }
}
```

6. Sample Output

One possible output (the thread order may differ):

```
Ravi requested 4 tickets.
Booking successful for Ravi
Tickets booked: 4
Seats remaining: 6
Priya requested 3 tickets.
Booking successful for Priya
Tickets booked: 3
Seats remaining: 3
Rahul requested 5 tickets.
Booking failed for Rahul
Not enough seats available.
Ticket booking completed.
```

7. Result

The program demonstrates synchronization successfully. The synchronized method prevents simultaneous updates to the shared seat count and keeps ticket booking consistent.

Compile and run:

```
javac -d . TicketBooking.java
java recordprograms.TicketBooking
```